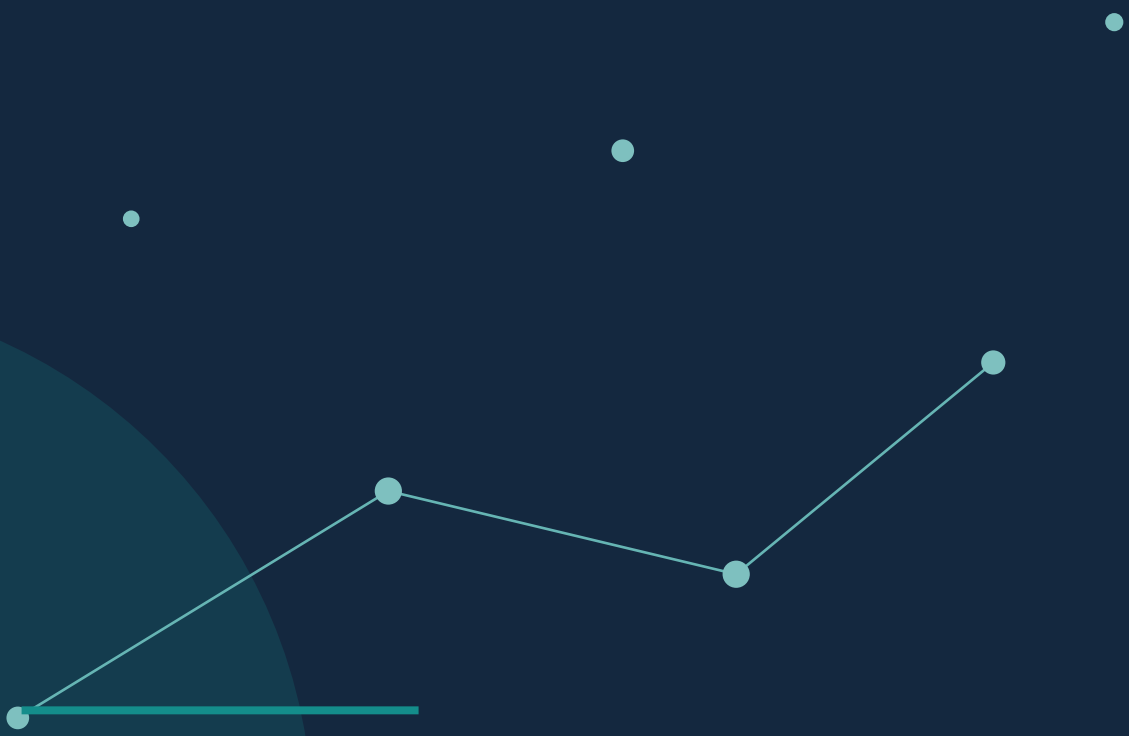
GUÍA ILUSTRADA · TEORÍA + EJEMPLOS + PYTHON

REDES NEURONALES ARTIFICIALES

De la intuición al funcionamiento matemático y su implementación en Python



Diseñada para lectores técnicos y no técnicos

Material de estudio ampliado a partir de la presentación suministrada

Octubre de 2026 · Versión 1.0

Antes de comenzar

Esta guía convierte los conceptos de una presentación sobre *Artificial Neural Networks* en un recorrido que puede leerse de principio a fin o consultarse por temas. No es necesario saber programar para comprender las primeras secciones. Los ejemplos matemáticos y de código permiten profundizar de forma progresiva.

La gran idea: una red neuronal transforma entradas en predicciones y mejora esas predicciones ajustando números llamados **pesos** a partir de ejemplos.

SI NO PROGRAMAS	SI PROGRAMAS O ESTUDIAS IA
Lee las analogías, los diagramas y las cajas azules. Comprenderás qué hace una red neuronal, por qué aprende y qué significan sus resultados.	Sigue las ecuaciones, las cajas verdes y los ejemplos en NumPy y PyTorch. Podrás reproducir una pasada hacia delante, calcular gradientes y entrenar un MLP.

Objetivos de aprendizaje

Al finalizar deberías poder:

1. Explicar qué representa una neurona artificial y cómo se organiza en capas.
2. Distinguir regresión, clasificación binaria y multiclase.
3. Reconocer el papel de las activaciones, la función de pérdida y la tasa de aprendizaje.
4. Seguir numéricamente una predicción y una actualización mediante *backpropagation*.
5. Leer, ejecutar y modificar ejemplos básicos de entrenamiento en Python.

ATENCIÓN

Alcance. Los casos, números y datos sintéticos de esta guía son didácticos. Un modelo que funciona en un ejercicio no está automáticamente preparado para tomar decisiones reales, médicas, financieras o de otro tipo.

Origen del material. La presentación de partida se titula *Artificial Neural Networks*, atribuida a Héctor J. Hortúa, PhD. Se preservan sus ejes conceptuales (neurona, activaciones, MLP, descenso de gradiente y retropropagación). Las explicaciones, diagramas, ejemplos y códigos de esta guía son ampliaciones didácticas; cuando se introducen matices modernos se señalan expresamente.

Índice

Antes de comenzar 1

1. ¿Qué es una red neuronal? 4
1.1. ¿Dónde aparece el aprendizaje? . . . 4

2. De una suma ponderada a una neurona 5
2.1. Ejemplo con dos entradas . . . 5

3. Cómo se construye una red 6
3.1. La notación matricial . . . 6

4. Funciones de activación 8
4.1. Softmax con tres clases . . . 8

5. ¿Qué problema estamos resolviendo? 10
5.1. Elegir un umbral no es entrenar . . . 10

6. La función de pérdida 11
6.1. Regresión: error cuadrático medio . . . 11
6.2. Clasificación binaria: entropía cruzada . . . 11
6.3. Clasificación multiclase: entropía cruzada categórica . . . 11

7. Propagación hacia delante 12
7.1. Paso 1. Primera neurona oculta . . . 12
7.2. Paso 2. Segunda neurona oculta . . . 12
7.3. Paso 3. Neurona de salida . . . 12
7.4. Comprobar la predicción en NumPy . . . 12

8. Retropropagación: el corazón del aprendizaje 14
8.1. Paso 4. Calcular la señal de error en la salida . . . 14
8.2. Paso 5. Llevar el error a la capa oculta . . . 14

9. Actualizar los pesos, paso a paso 15
9.1. Paso 6. Actualizar los pesos hacia la salida . . . 15
9.2. Paso 7. Actualizar los pesos de entrada . . . 15
9.3. Código que reproduce la actualización . . . 15

10. Descenso de gradiente 17
10.1. Tres maneras de calcular el gradiente . . . 17

11. Tasa de aprendizaje y dificultades 18

11.1. Superficies de pérdida no convexas . . . 18
11.2. Problemas de entrenamiento frecuentes . . . 19

12. Preparar los datos correctamente 20
12.1. Separar entrenamiento, validación y prueba . . . 20
12.2. Escalado sin fuga de información . . . 20

13. Primer modelo desde cero: NumPy 22
13.1. Paso 1. Crear datos de ejemplo . . . 22
13.2. Paso 2. Definir función, pérdida y gradientes . . . 22
13.3. Paso 3. Repetir el aprendizaje . . . 22
13.4. Qué puedes modificar y observar . . . 23

14. Una red real con PyTorch 24
14.1. Paso 1. Datos y división . . . 24
14.2. Paso 2. Definir la arquitectura y el optimizador . . . 24
14.3. Paso 3. Ejecutar el ciclo de entrenamiento . . . 24
14.4. Paso 4. Evaluar en datos reservados . . . 25

15. Qué ocurre dentro de PyTorch 26
15.1. Diferenciar tres operaciones . . . 26
15.2. ¿Para qué sirve `DataLoader`? . . . 26

16. ¿Cómo saber si el modelo funciona? 27
16.1. Las cuatro salidas de una clasificación binaria . . . 27
16.2. Errores frecuentes al interpretar un modelo . . . 27

17. Dónde se usan y qué límites tienen 28
17.1. Tres preguntas para evaluar una propuesta . . . 28

18. Laboratorio guiado y ejercicios 29
18.1. Ejercicio 1. Interpretación sin programación . . . 29
18.2. Ejercicio 2. Cálculo de una neurona . . . 29
18.3. Ejercicio 3. Arquitecturas . . . 29
18.4. Ejercicio 4. Detectar un error de código . . . 29
18.5. Ejercicio 5. Interpretación de métricas . . . 29
18.6. Ejercicio 6. Miniinvestigación . . . 29

19. Glosario visual de referencia 31
19.1. La ruta de aprendizaje en seis preguntas . . . 31

20. Bibliografía y material complementario 32

1 ¿Qué es una red neuronal?

Una explicación que funciona sin fórmulas

Una **red neuronal artificial** es un modelo computacional compuesto por unidades sencillas conectadas entre sí. Cada unidad recibe información, la combina y entrega un resultado. Cuando muchas unidades trabajan en capas, la red puede modelar relaciones complejas entre variables.

EN LENGUAJE COTIDIANO

Imagina que quieres reconocer si una fotografía contiene un ave. Puedes fijarte en el pico, la forma de las alas, el plumaje y el contorno. Una red neuronal no recibe necesariamente esas reglas escritas a mano: aprende, a partir de ejemplos etiquetados, qué combinaciones de características ayudan a distinguir las clases.

LECTURA TÉCNICA

Una red parametrizada representa una función $f_\theta : \mathcal{X} \to \mathcal{Y}$, donde θ agrupa pesos y sesgos. El entrenamiento busca parámetros θ que minimicen una pérdida promedio sobre los ejemplos observados, con la intención de generalizar a datos nuevos.

1.1 ¿Dónde aparece el aprendizaje?

1. **Predicción:** la red recibe un ejemplo y produce una salida.
2. **Comparación:** mide cuánto se diferencia la salida de la respuesta esperada.
3. **Corrección:** calcula qué parámetros contribuyeron al error.
4. **Repetición:** modifica gradualmente los parámetros y vuelve a intentarlo.

El paralelismo con las neuronas biológicas es una inspiración histórica, no una reproducción fiel del cerebro. Una red neuronal es, en términos operativos, una función matemática entrenable.

LO ESENCIAL

Datos $\to$ **predicción** $\to$ **error** $\to$ **ajuste**. Aprender no significa memorizar respuestas: lo deseable es descubrir regularidades útiles para ejemplos que el modelo todavía no ha visto.

2 De una suma ponderada a una neurona

La relación con la regresión lineal y logística

Una neurona artificial realiza dos operaciones: combina entradas con pesos y aplica, si corresponde, una función de activación.

$$\boxed{z = w_1x_1 + w_2x_2 + \cdots + w_mx_m + b} \qquad \boxed{a = \phi(z)} \tag{1}$$

Símbolo	Interpretación
$x_1, \ldots, x_m$	Características o variables de entrada.
$w_1, \ldots, w_m$	Pesos aprendidos: determinan la influencia de cada entrada.
b	Sesgo o *bias*: desplaza el resultado, incluso si todas las entradas son cero.
z	Combinación lineal previa a la activación.
ϕ	Función de activación, como sigmoide o ReLU.
a	Valor que entrega la neurona.

EN LENGUAJE COTIDIANO

Piensa en una receta. Los ingredientes son las entradas; la cantidad de cada ingrediente representa el peso; el sesgo permite modificar el sabor de base. La activación decide cómo transformar el resultado de esa mezcla antes de enviarlo a la siguiente etapa.

2.1 Ejemplo con dos entradas

Supongamos $x_1 = 0{,}35$, $x_2 = 0{,}70$, $w_1 = 0{,}20$, $w_2 = 0{,}30$ y $b = -0{,}10$.

$$z = (0{,}20)(0{,}35) + (0{,}30)(0{,}70) - 0{,}10 = \boxed{0{,}18}.$$

Si usamos la **sigmoide**, $\sigma(z) = 1/(1 + e^{-z})$, la salida es aproximadamente $\sigma(0{,}18) = \boxed{0{,}545}$.

LECTURA TÉCNICA

La regresión lineal usa una salida lineal ($\hat{y} = z$); la regresión logística binaria aplica $\sigma(z)$ para producir un valor entre 0 y 1. Una neurona aislada con activación sigmoide tiene, por tanto, la forma matemática del modelo logístico binario. Una red multicapa combina muchas de estas transformaciones.

```python
import numpy as np

x = np.array([0.35, 0.70])
w = np.array([0.20, 0.30])
b = -0.10
z = np.dot(w, x) + b
prob = 1 / (1 + np.exp(-z))
print(round(z, 3), round(prob, 3))  # 0.18 0.545
```

3 Cómo se construye una red

Capas, conexiones, arquitectura y tensores

Una red multicapa agrupa neuronas en **capa de entrada**, **capas ocultas** y **capa de salida**. La información avanza de una capa a otra durante la predicción.

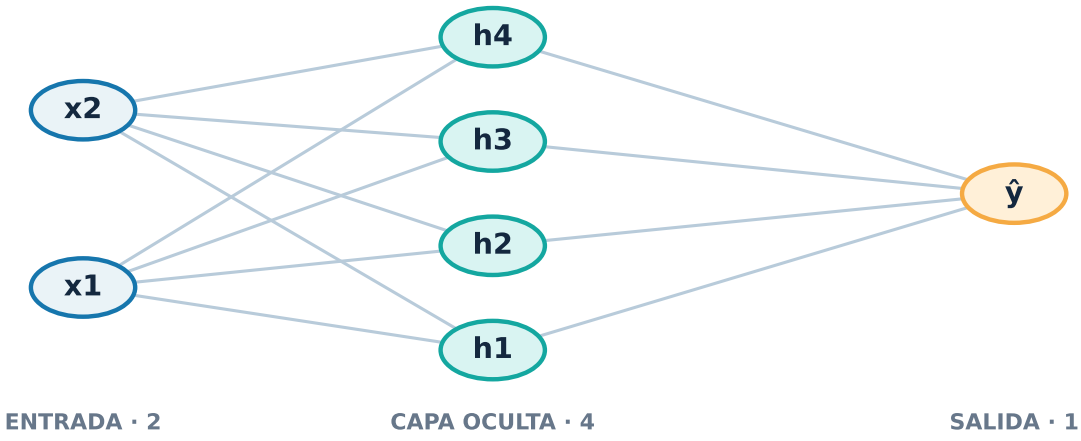


Figura 1: Arquitectura ilustrativa $2 \to 4 \to 1$: dos entradas, cuatro neuronas ocultas y una salida. Las conexiones contienen pesos entrenables.

EN LENGUAJE COTIDIANO

Una persona puede detectar líneas en una fotografía; otra combina líneas para reconocer plumas; una tercera reúne las pistas y produce una etiqueta. Cada capa transforma lo que recibió para facilitar la tarea de la capa siguiente. En redes reales, esas representaciones se aprenden automáticamente.

3.1 La notación matricial

Para una capa de m neuronas que recibe d entradas, escribimos

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \qquad \mathbf{a} = \phi(\mathbf{z}),$$

donde $W \in \mathbb{R}^{m \times d}$, $\mathbf{x} \in \mathbb{R}^{d}$, $\mathbf{b} \in \mathbb{R}^{m}$ y $\mathbf{a} \in \mathbb{R}^{m}$.

LECTURA TÉCNICA

Una arquitectura $2 \to 4 \to 1$ tiene $W^{(1)}$ de forma $(4, 2)$, $b^{(1)}$ de forma $(4,)$, $W^{(2)}$ de forma $(1, 4)$ y $b^{(2)}$ de forma $(1,)$. El total es $8 + 4 + 4 + 1 = \boxed{17}$ parámetros entrenables. Con un lote de B ejemplos, la matriz de entrada normalmente tiene forma $(B, 2)$.

```
# Una pasada hacia delante de 5 registros, cada uno con 2 variables
X = np.random.default_rng(7).normal(size=(5, 2))
W1 = np.random.default_rng(8).normal(size=(2, 4))
b1 = np.zeros(4)
W2 = np.random.default_rng(9).normal(size=(4, 1))
b2 = np.zeros(1)
H = np.maximum(X @ W1 + b1, 0)  # ReLU; forma (5, 4)
```

```
Y = H @ W2 + b2                # forma (5, 1)
print(X.shape, H.shape, Y.shape)
```

ATENCIÓN

Una capa con activación lineal tras otra capa lineal se puede reducir a una sola transformación lineal. Las funciones de activación **no lineales** permiten representar patrones más expresivos.

4 Funciones de activación

Cuándo usar lineal, sigmoide, tanh, ReLU y softmax

La activación determina cómo se transforma la señal en cada neurona y condiciona qué relaciones puede aprender la red.

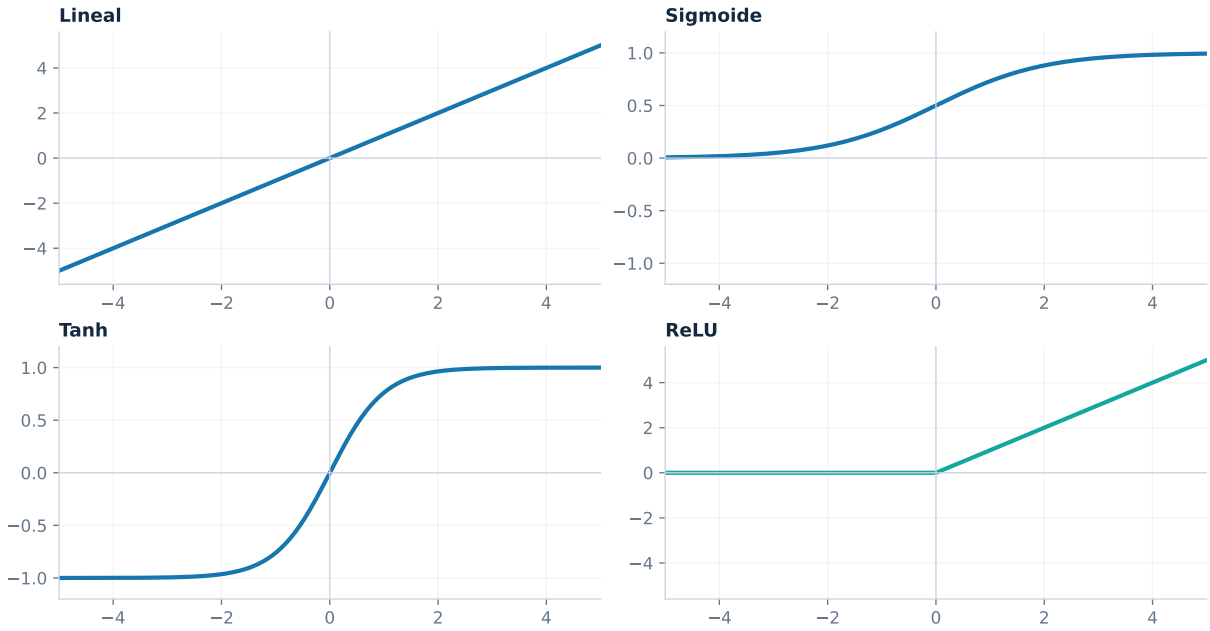


Figura 2: Formas típicas de cuatro funciones de activación. Los valores del eje vertical son diferentes según la función.

Función	Expresión	Uso y observación
Lineal	$\phi(z) = z$	Salida habitual en regresión sin restricción.
Sigmoide	$\sigma(z) = \frac{1}{1+e^{-z}}$	Salida binaria: valor entre 0 y 1. Puede saturarse.
Tanh	$\tanh(z)$	Salida entre -1 y 1; útil en ciertas capas ocultas.
ReLU	$\max(0, z)$	Activación frecuente en capas ocultas; simple y eficiente.
Softmax	$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$	Convierte logits de varias clases excluyentes en probabilidades que suman 1.

LECTURA TÉCNICA

Las derivadas explican parte de su comportamiento: $\sigma'(z) = \sigma(z)(1-\sigma(z))$; $\tanh'(z) = 1-\tanh^2(z)$; para ReLU la derivada vale 0 si $z < 0$ y 1 si $z > 0$ (en cero se elige una subderivada). Las saturaciones de sigmoide y tanh pueden reducir la magnitud de los gradientes.

4.1 Softmax con tres clases

Si los **logits** (puntuaciones sin normalizar) son $(2, 1, 0{,}1)$, softmax produce aproximadamente:

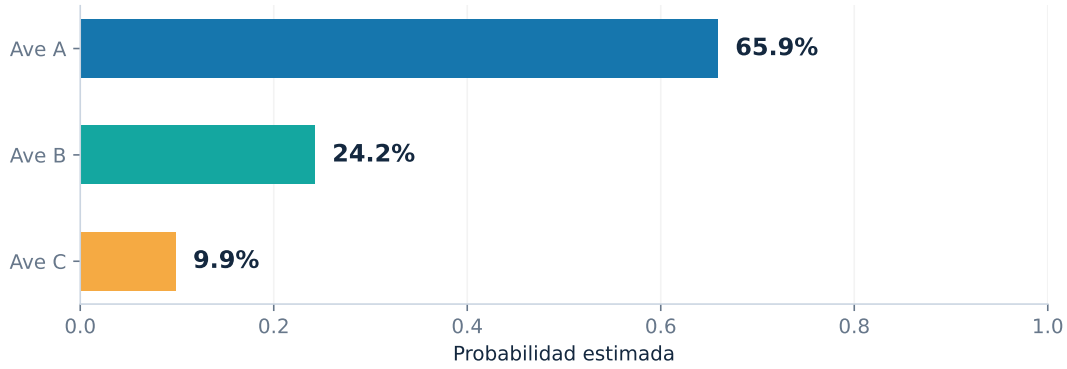


Figura 3: Ejemplo ilustrativo de softmax: los tres valores están entre 0 y 1 y suman 100 %.

```
logits = np.array([2.0, 1.0, 0.1])
e = np.exp(logits - logits.max())  # estabilidad numerica
p = e / e.sum()
print(np.round(p, 3))  # [0.659 0.242 0.099]
```

ATENCIÓN

Importante: una salida de 0.66 no demuestra por sí sola que el modelo esté bien calibrado ni que tenga 66 % de probabilidad de acertar en ese caso. La calidad probabilística debe evaluarse por separado.

5 ¿Qué problema estamos resolviendo?

Regresión, clasificación binaria y clasificación multiclase

La capa de salida y la función de pérdida se eligen en función del tipo de respuesta que deseamos predecir.

Problema	Ejemplo	Salida típica	Pérdida típica
Regresión	Estimar un consumo eléctrico.	1 o más valores lineales.	Error cuadrático medio (MSE).
Binaria	Detectar si una transacción es sospechosa (sí/no).	1 logit; sigmoide para interpretar.	Entropía cruzada binaria.
Multiclase	Reconocer una especie entre 4 opciones.	4 logits; softmax para interpretar.	Entropía cruzada categórica.
Multietiqueta	Asignar varios atributos a una imagen.	Una salida independiente por etiqueta.	Entropía cruzada binaria por etiqueta.

EN LENGUAJE COTIDIANO

Regresión: «¿Cuánto?». **Binaria:** «¿Sí o no?». **Multiclase:** «¿Cuál de las opciones?». **Multietiqueta:** «¿Cuáles etiquetas aplican al mismo tiempo?». No son intercambiables, aunque puedan utilizar capas ocultas parecidas.

LECTURA TÉCNICA

La *regresión logística multinomial* permite clasificar más de dos clases usando softmax. Por eso, la afirmación de que «la regresión logística solo sirve para dos clases» es válida únicamente cuando hablamos de su formulación **binaria**, no de toda la familia de modelos logísticos.

5.1 Elegir un umbral no es entrenar

En clasificación binaria, si el modelo produce 0.73, un umbral de 0.50 clasifica como positivo. Modificar el umbral cambia las decisiones, la sensibilidad y la especificidad, pero **no cambia los pesos aprendidos**. El umbral apropiado depende del costo de falsos positivos y falsos negativos.

LO ESENCIAL

Primero identifica la naturaleza del objetivo. **Después** escoge la salida y la pérdida compatibles. Si la salida y la pérdida no corresponden al problema, el entrenamiento puede ser incorrecto aunque el código se ejecute.

6 La función de pérdida

Cómo expresar matemáticamente el error

Durante el entrenamiento hay un valor deseado y y una predicción $\hat{y}$. La función de pérdida $\mathcal{L}(y, \hat{y})$ mide la discrepancia que queremos reducir.

6.1 Regresión: error cuadrático medio

Para n ejemplos:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2.$$

Si el valor real es 10 y la predicción es 8, la contribución al MSE es $(8-10)^2 = 4$.

6.2 Clasificación binaria: entropía cruzada

$$\text{BCE} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right].$$

Una predicción incorrecta expresada con gran seguridad recibe un castigo alto. Por ejemplo, si $y = 1$, declarar $p = 0{,}01$ genera una pérdida mayor que declarar $p = 0{,}45$.

6.3 Clasificación multiclase: entropía cruzada categórica

Si la clase verdadera es k y p_k es su probabilidad predicha:

$$\mathcal{L} = -\log(p_k).$$

Esto premia asignar mucha probabilidad a la clase correcta. En implementaciones numéricamente estables se utilizan directamente los **logits**.

LECTURA TÉCNICA

En PyTorch, `BCEWithLogitsLoss` combina sigmoide y BCE de forma estable. `CrossEntropyLoss` recibe logits crudos para clasificación multiclase y aplica internamente una operación equivalente a *log-softmax* y pérdida negativa del logaritmo. **No** agregues otra sigmoide o softmax antes de esas pérdidas.

```
import torch
binary_logits = torch.tensor([[0.4], [-1.2]])
binary_y = torch.tensor([[1.0], [0.0]])
loss = torch.nn.BCEWithLogitsLoss()(binary_logits, binary_y)
print('BCE:', round(loss.item(), 4))
```

7 Propagación hacia delante

Una predicción completa, con números reales del ejercicio

Retomamos el ejercicio numérico de la presentación original: dos entradas, dos neuronas ocultas y una neurona de salida, todas con sigmoide y sesgos fijados en cero para simplificar. Los pesos iniciales son:

$$\mathbf{x} = \begin{bmatrix} 0{,}35 \\ 0{,}70 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0{,}20 & 0{,}20 \\ 0{,}30 & 0{,}30 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0{,}30 & 0{,}90 \end{bmatrix}, \quad y = 0{,}50.$$

EN LENGUAJE COTIDIANO

La red todavía no «sabe» cuál es la salida correcta. Primero combina las dos entradas para producir señales intermedias y luego combina esas señales para obtener una predicción. Después podremos comparar esa predicción con el objetivo 0.50.

7.1 Paso 1. Primera neurona oculta

$$z_3 = 0{,}20(0{,}35) + 0{,}20(0{,}70) = 0{,}2100, \quad h_3 = \sigma(z_3) = \boxed{0{,}5523}.$$

7.2 Paso 2. Segunda neurona oculta

$$z_4 = 0{,}30(0{,}35) + 0{,}30(0{,}70) = 0{,}3150, \quad h_4 = \sigma(z_4) = \boxed{0{,}5781}.$$

7.3 Paso 3. Neurona de salida

$$z_5 = 0{,}30(0{,}5523) + 0{,}90(0{,}5781) \approx 0{,}6860,$$

$$\hat{y} = \sigma(z_5) = \boxed{0{,}6651}.$$

La red predice 0.6651 frente a un objetivo de 0.50. Por tanto, el residuo $\hat{y} - y$ es aproximadamente $\boxed{+0{,}1651}$. Si usamos $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$, la pérdida es $\boxed{0{,}01362}$.

LECTURA TÉCNICA

El objetivo $y = 0{,}50$ y la pérdida cuadrática se utilizan **para reproducir un ejercicio de derivación**, no para proponer el diseño habitual de un clasificador binario. Para clasificación binaria real, la combinación de logits y BCE suele ser más adecuada.

7.4 Comprobar la predicción en NumPy

```
import numpy as np
sigmoid = lambda z: 1.0 / (1.0 + np.exp(-z))
x = np.array([0.35, 0.70]); y = 0.50
W1 = np.array([[0.20, 0.20], [0.30, 0.30]])
W2 = np.array([0.30, 0.90])
```

```
h = sigmoid(W1 @ x)
y_pred = sigmoid(W2 @ h)
loss = 0.5 * (y_pred - y)**2
print(np.round(h, 4), round(y_pred, 4), round(loss, 5))
```

8 Retropropagación: el corazón del aprendizaje

Cómo asignar responsabilidades al error

La **retropropagación** (*backpropagation*) aplica la regla de la cadena para calcular derivadas de la pérdida respecto de cada peso. Después, un optimizador utiliza esos gradientes para modificar los pesos.

EN LENGUAJE COTIDIANO

Si una receta queda demasiado dulce, no basta con decir «salió mal». Hay que averiguar qué ingredientes y cantidades influyeron en el resultado. En una red, la retropropagación calcula cuánto contribuyó cada parámetro a la pérdida, empezando por la salida y retrocediendo capa por capa.

8.1 Paso 4. Calcular la señal de error en la salida

Con $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$ y activación sigmoide:

$$\delta_5 = \frac{\partial \mathcal{L}}{\partial z_5} = (\hat{y} - y)\,\hat{y}\,(1 - \hat{y}).$$

Insertando los valores:

$$\delta_5 = (0{,}6651 - 0{,}50)(0{,}6651)(1 - 0{,}6651) \approx \boxed{0{,}03677}.$$

8.2 Paso 5. Llevar el error a la capa oculta

Para cada neurona oculta j:

$$\delta_j = h_j(1 - h_j)w_{j5}\delta_5.$$

$$\delta_3 \approx 0{,}5523(0{,}4477)(0{,}30)(0{,}03677) = \boxed{0{,}002728},$$

$$\delta_4 \approx 0{,}5781(0{,}4219)(0{,}90)(0{,}03677) = \boxed{0{,}008071}.$$

LECTURA TÉCNICA

La regla de la cadena enlaza las derivadas locales $\partial\mathcal{L}/\partial\hat{y}$, $\partial\hat{y}/\partial z_5$ y $\partial z_5/\partial w$. Los gradientes de los pesos de salida son $\partial\mathcal{L}/\partial w_{j5} = \delta_5 h_j$. Los gradientes de entrada a oculta son $\partial\mathcal{L}/\partial w_{ij} = \delta_j x_i$.

ATENCIÓN

En algunas diapositivas se define δ con el signo $y - \hat{y}$ y se **suma** la corrección. Aquí se usa δ como gradiente con signo $\hat{y} - y$ y se **resta** la corrección. Ambas convenciones pueden ser equivalentes si se aplican de forma coherente.

9 Actualizar los pesos, paso a paso

Cierre numérico del ejercicio de backpropagation

Una vez calculados los gradientes, el **descenso de gradiente** utiliza

$$\boxed{w_{\text{nuevo}} = w_{\text{anterior}} - \eta \frac{\partial \mathcal{L}}{\partial w}},$$

donde $\eta > 0$ es la **tasa de aprendizaje**. Para reproducir el ejercicio tomamos $\eta = 1$ (solo para mostrar las operaciones).

9.1 Paso 6. Actualizar los pesos hacia la salida

$$w_{35}^{\text{nuevo}} = 0{,}30 - (1)(0{,}0367703)(0{,}5523079) = \boxed{0{,}2796915},$$
$$w_{45}^{\text{nuevo}} = 0{,}90 - (1)(0{,}0367703)(0{,}5781052) = \boxed{0{,}8787429}.$$

9.2 Paso 7. Actualizar los pesos de entrada

Para los pesos conectados con h_3 y h_4 usamos $w_{ij}^{nuevo} = w_{ij} - \eta \delta_j x_i$.

Peso	Valor inicial	Valor actualizado
w_{13}	0.20000	**0.19905**
w_{23}	0.20000	**0.19809**
w_{14}	0.30000	**0.29718**
w_{24}	0.30000	**0.29435**
w_{35}	0.30000	**0.27969**
w_{45}	0.90000	**0.87874**

EN LENGUAJE COTIDIANO

Los seis pesos disminuyen porque, para este ejemplo concreto, sus gradientes son positivos y la salida inicial era más alta que el objetivo. En otros ejemplos los cambios pueden ser positivos o negativos. **No** existe una regla general que diga que todos los pesos deban disminuir.

9.3 Código que reproduce la actualización

```
delta_out = (y_pred - y) * y_pred * (1 - y_pred)
delta_h = h * (1 - h) * W2 * delta_out
eta = 1.0
W2_new = W2 - eta * delta_out * h
W1_new = W1 - eta * np.outer(delta_h, x)
print('W2:', np.round(W2_new, 5))
print('W1:\n', np.round(W1_new, 5))
```

LO ESENCIAL

Forward: obtengo $\hat{y}$. **Pérdida:** cuantifico el error. **Backward:** calculo derivadas. **Optimizador:** actualizo los parámetros. **Iteración:** repito el ciclo con otros ejemplos o lotes.

10 Descenso de gradiente

Por qué optimizar se parece a bajar una montaña

Considera que cada combinación de pesos corresponde a un punto en una superficie, cuya altura es la pérdida. Aprender equivale a buscar regiones donde esa altura sea pequeña.

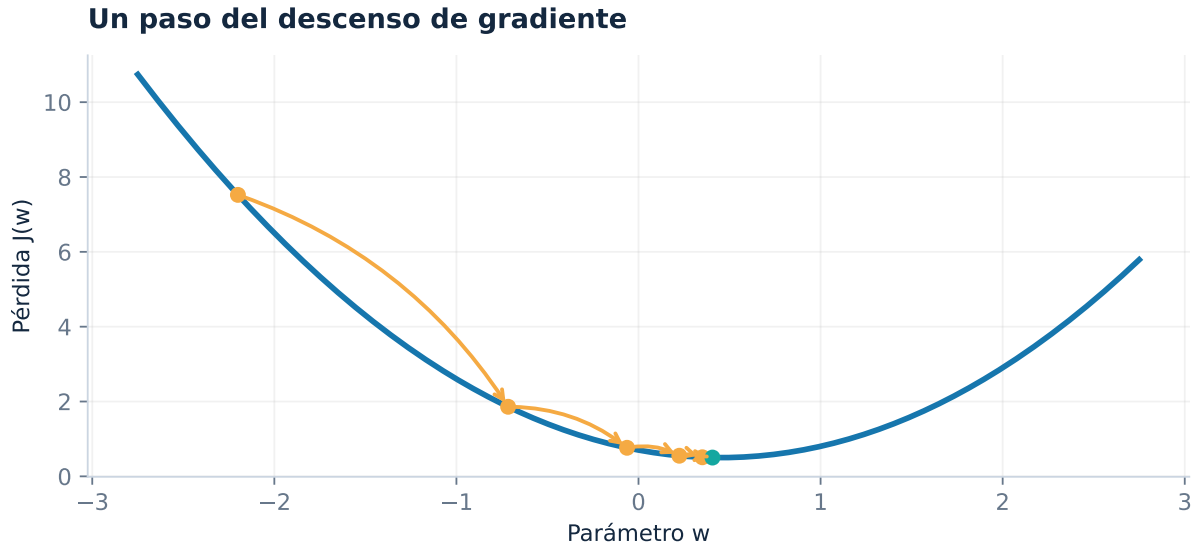


Figura 4: Descenso de gradiente en un ejemplo convexo de una dimensión. Las flechas muestran actualizaciones sucesivas hacia la región de menor pérdida.

EN LENGUAJE COTIDIANO

Una persona que desciende una montaña con niebla no observa todo el terreno. Detecta la pendiente local y avanza en la dirección que permite bajar. El gradiente señala la dirección de mayor crecimiento local; el signo menos indica cómo disminuir la pérdida.

10.1 Tres maneras de calcular el gradiente

Método	Cómo funciona
Por lote completo	Calcula el gradiente usando todos los ejemplos de entrenamiento antes de cada actualización.
Estocástico (SGD)	Actualiza a partir de un ejemplo (o de muestras unitarias). Tiene más variación entre pasos.
Por minilotes	Usa pequeños grupos de ejemplos; suele equilibrar costo computacional y estabilidad.

LECTURA TÉCNICA

Si θ es un vector con todos los parámetros, la actualización básica es $\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$. La retropropagación *calcula* $\nabla_\theta \mathcal{L}$; SGD y Adam son algoritmos que *usan* esos gradientes para actualizar θ. No son sinónimos.

11 Tasa de aprendizaje y dificultades

Mínimos locales, puntos de silla y mesetas

La tasa de aprendizaje η controla el tamaño de cada paso en el espacio de parámetros.

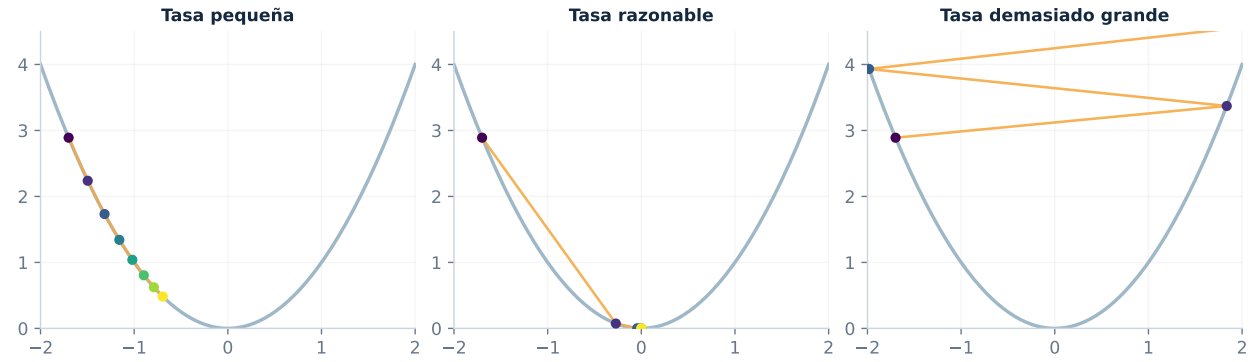


Figura 5: Con una tasa pequeña el avance puede ser lento; una tasa grande puede provocar oscilación o divergencia. La zona adecuada depende del problema y la escala.

11.1 Superficies de pérdida no convexas

Una red neuronal multicapa tiene, por lo general, una superficie de optimización no convexa. Pueden aparecer mínimos locales, regiones casi planas y **puntos de silla**.

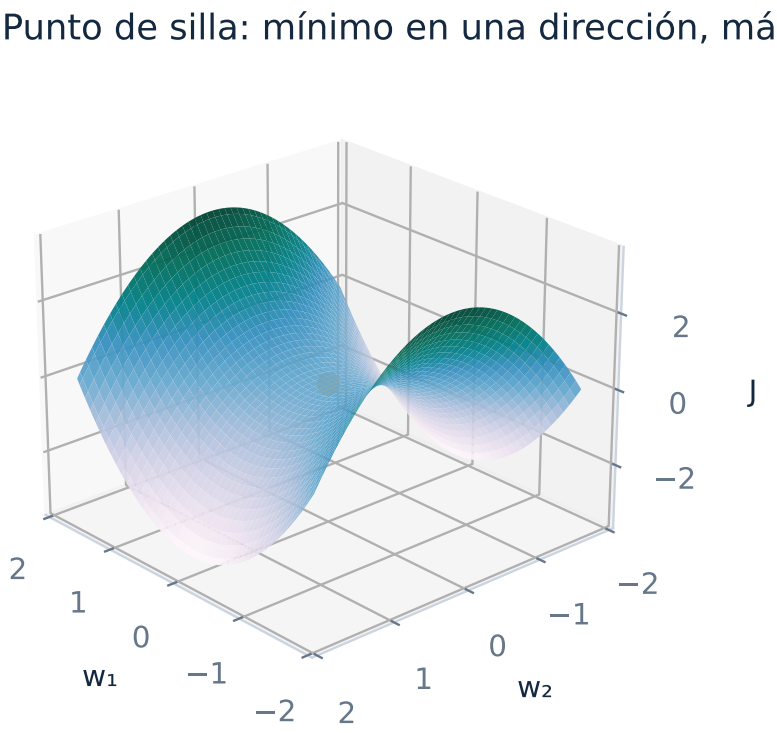


Figura 6: Ejemplo geométrico de punto de silla: el mismo punto se comporta como mínimo en una dirección y máximo en otra.

LECTURA TÉCNICA

La función cuadrática de regresión lineal con MSE es convexa respecto a sus coeficientes; en cambio, las redes profundas suelen presentar geometrías mucho más complejas. No hay garantía general de que un algoritmo local encuentre el mínimo global. Sin embargo, una solución con buena generalización puede ser suficiente en la práctica.

11.2 Problemas de entrenamiento frecuentes

- **Gradientes pequeños:** las actualizaciones casi no avanzan; puede intervenir la saturación de activaciones.
- **Gradientes grandes:** el entrenamiento se vuelve inestable; son útiles la normalización, una tasa menor o *gradient clipping*.
- **Sobreajuste:** mejora el conjunto de entrenamiento, pero empeora el desempeño en ejemplos nuevos.
- **Aprendizaje insuficiente:** la red no captura el patrón; quizá la arquitectura o el entrenamiento sean inadecuados.

ATENCIÓN

No elijas la tasa de aprendizaje solo porque otra persona utilizó 0.001 o 0.1. Prueba varias escalas y observa tanto la curva de entrenamiento como la de validación. En el ejemplo manual se usó $\eta = 1$ por conveniencia aritmética, no como recomendación universal.

12 Preparar los datos correctamente

Qué ocurre antes de entrenar la red

Un buen modelo comienza mucho antes de la primera multiplicación matricial. La preparación de los datos es una parte esencial de la calidad del resultado.

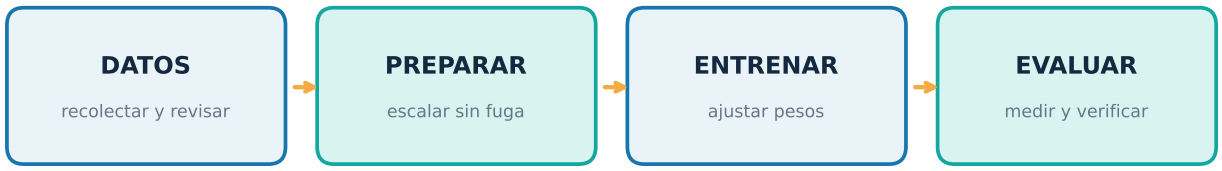


Figura 7: Flujo mínimo recomendado para un experimento supervisado.

12.1 Separar entrenamiento, validación y prueba

Partición	Para qué se utiliza
Entrenamiento	Ajustar pesos y demás parámetros aprendidos por el modelo.
Validación	Elegir hiperparámetros y detectar sobreajuste.
Prueba	Estimar el desempeño final en datos no utilizados para tomar decisiones de entrenamiento.

EN LENGUAJE COTIDIANO

Preparar un examen con preguntas de práctica ayuda a aprender; usar esas mismas preguntas como examen final no permite medir bien cuánto se puede responder sobre material nuevo. Algo similar ocurre si se evalúa el modelo en los mismos datos con los que fue ajustado.

12.2 Escalado sin fuga de información

Una estandarización típica consiste en $z = (x - \mu_{train})/s_{train}$. Calcula media y desviación **solo con entrenamiento**; luego reutilízalas sin volverlas a ajustar en validación y prueba.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
scaler = StandardScaler().fit(X_tr)  # solo train
X_tr = scaler.transform(X_tr)
X_te = scaler.transform(X_te)
```

ATENCIÓN

Fuga de datos (*data leakage*): sucede cuando alguna información de validación o prueba afecta el entrenamiento. También hay fugas cuando una misma persona, sujeto o unidad aparece en ambas particiones; puede requerirse división por grupos o por tiempo.

13 Primer modelo desde cero: NumPy

Una neurona logística que aprende a clasificar

Construiremos un conjunto sintético de dos variables y una etiqueta binaria. El propósito es observar los cálculos de entrenamiento sin apoyarnos en una biblioteca de redes neuronales.

13.1 Paso 1. Crear datos de ejemplo

```
import numpy as np
rng = np.random.default_rng(42)
X = rng.normal(size=(400, 2))
true_w = np.array([1.5, -2.0])
logits = X @ true_w + 0.4 + rng.normal(0, 0.5, 400)
y = (logits > 0).astype(float)
# X tiene escala cercana a 1 por construccion.
```

13.2 Paso 2. Definir función, pérdida y gradientes

Sea $p_i = \sigma(\mathbf{x}_i^T \mathbf{w} + b)$. Para la entropía cruzada binaria promedio, los gradientes simplificados son

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{n} X^T (\mathbf{p} - \mathbf{y}), \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i).$$

```
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def bce(y, p):
    p = np.clip(p, 1e-12, 1 - 1e-12)
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))

w = np.zeros(2)
b = 0.0
history = []
```

13.3 Paso 3. Repetir el aprendizaje

```
rate = 0.2
for epoch in range(600):
    p = sigmoid(X @ w + b)
    history.append(bce(y, p))
    grad_w = X.T @ (p - y) / len(y)
    grad_b = np.mean(p - y)
    w -= rate * grad_w
    b -= rate * grad_b

pred = (sigmoid(X @ w + b) >= 0.5).astype(float)
print('Accuracy en datos sintéticos:', round(np.mean(pred == y), 3))
print('Pesos:', np.round(w, 3), 'sesgo:', round(b, 3))
```

LECTURA TÉCNICA

Esta demostración usa todos los registros para ajustar y medir en el mismo conjunto, **solo** para mostrar los gradientes y el proceso de aprendizaje. No constituye una evaluación válida de generalización. El siguiente ejemplo separa entrenamiento y prueba.

13.4 Qué puedes modificar y observar

Prueba una tasa de aprendizaje de $0{,}01$, $0{,}2$ y $2{,}0$; compara la pérdida inicial y final. Cambia el nivel de ruido de 0.5 a 2.0 y observa cómo un problema menos separable limita el desempeño.

14 Una red real con PyTorch

Clasificación no lineal, DataLoader y entrenamiento

Ahora usamos un conjunto de dos grupos con forma de medialuna; una frontera lineal no suele capturar bien su estructura. Un **perceptrón multicapa** (MLP) con una capa oculta de 16 neuronas puede aprender una frontera no lineal.

14.1 Paso 1. Datos y división

```
import numpy as np
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

torch.manual_seed(42)
X, y = make_moons(n_samples=1000, noise=0.20, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
scaler = StandardScaler().fit(X_tr)
X_tr, X_te = scaler.transform(X_tr), scaler.transform(X_te)
train_ds = TensorDataset(torch.tensor(X_tr, dtype=torch.float32),
                         torch.tensor(y_tr, dtype=torch.float32).unsqueeze(1))
loader = DataLoader(train_ds, batch_size=32, shuffle=True)
```

14.2 Paso 2. Definir la arquitectura y el optimizador

```
model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 1)  # devuelve logits, NO probabilidades
)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

LECTURA TÉCNICA

`nn.Linear(2, 16)` aprende 16×2 pesos y 16 sesgos. La salida aprende 16 pesos y 1 sesgo: en total $32 + 16 + 16 + 1 = \boxed{65}$ parámetros. La función de pérdida consume logits y se encarga de la estabilidad numérica.

14.3 Paso 3. Ejecutar el ciclo de entrenamiento

```
for epoch in range(150):
    model.train()
    for xb, yb in loader:
        optimizer.zero_grad()        # limpia gradientes anteriores
```

```
        logits = model(xb)           # forward
        loss = criterion(logits, yb) # perdida
        loss.backward()             # backpropagation
        optimizer.step()             # actualizacion
```

14.4 Paso 4. Evaluar en datos reservados

```
model.eval()
with torch.no_grad():
    xt = torch.tensor(X_te, dtype=torch.float32)
    prob = torch.sigmoid(model(xt)).numpy().ravel()
    pred = (prob >= 0.5).astype(int)
accuracy = (pred == y_te).mean()
print(f'Accuracy de prueba: {accuracy:.3f}')
```

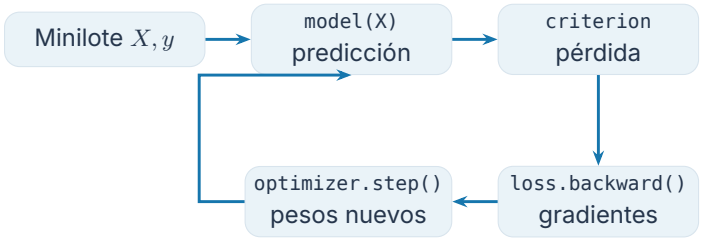
ATENCIÓN

El valor final de accuracy depende de la semilla, el conjunto de datos, las versiones de librerías y el entrenamiento. El código reproduce una **demostración**, no una promesa de desempeño ni una garantía de reproducibilidad bit a bit en todos los equipos.

15 Qué ocurre dentro de PyTorch

Tensores, gradientes automáticos y flujo de una época

PyTorch automatiza la derivación de las operaciones que participan en la pérdida mediante un sistema llamado *autograd*. No elimina el cálculo del gradiente: construye una forma eficiente de aplicarlo.



15.1 Diferenciar tres operaciones

Código	Función
`optimizer.zero_grad()`	Evita que los gradientes se acumulen entre lotes por accidente.
`loss.backward()`	Obtiene gradientes a través del grafo de cálculo.
`optimizer.step()`	Actualiza parámetros mediante el optimizador seleccionado.

LECTURA TÉCNICA

Durante la evaluación se utiliza `model.eval()` para cambiar el comportamiento de capas como *dropout* y *batch normalization*. `torch.no_grad()` evita crear el grafo de derivación y ahorra memoria. `eval()` y `no_grad()` resuelven problemas diferentes y suelen emplearse juntos.

15.2 ¿Para qué sirve `DataLoader`?

Permite recorrer el conjunto de datos en lotes, mezclarlos cuando corresponda y separar el almacenamiento de los datos de la lógica de entrenamiento. Si el conjunto tiene 800 registros y `batch_size=32`, cada época tiene 25 minilotes (si ninguno se descarta).

EN LENGUAJE COTIDIANO

Una **época** significa que el modelo tuvo la oportunidad de recorrer una vez todos los ejemplos de entrenamiento. Un **minilote** es una pequeña porción de esos ejemplos; normalmente ocurre una actualización de pesos por cada minilote.

16 ¿Cómo saber si el modelo funciona?

Métricas, interpretación y generalización

Que la pérdida disminuya no basta. Es necesario evaluar si el modelo alcanza el objetivo del problema y si el resultado se mantiene en datos nuevos.

16.1 Las cuatro salidas de una clasificación binaria

	Real positivo	**Real negativo**
Predicho positivo	Verdadero positivo (VP)	Falso positivo (FP)
Predicho negativo	Falso negativo (FN)	Verdadero negativo (VN)

$$\begin{aligned}
\text{Exactitud } (accuracy) &= \frac{VP+VN}{VP+VN+FP+FN},\\
\text{Precisión} &= \frac{VP}{VP+FP},\\
\text{Sensibilidad } (recall) &= \frac{VP}{VP+FN},\\
F_1 &= 2\frac{\text{Precisión}\cdot\text{Recall}}{\text{Precisión}+\text{Recall}}.
\end{aligned}$$

Estas métricas deben calcularse con cuidado cuando algún denominador sea cero.

EN LENGUAJE COTIDIANO

Si solo 1 de cada 100 transacciones es fraudulenta, predecir «no fraudulenta» siempre produce 99 % de accuracy, pero no detecta ningún fraude. Por eso una métrica alta puede ser engañosa cuando las clases están desbalanceadas.

LECTURA TÉCNICA

En escenarios desbalanceados suele ser conveniente revisar matrices de confusión, sensibilidad, precisión, curvas precisión-recall, evaluación por subgrupos e intervalos de incertidumbre. Para probabilidades, complementa con calibración y pérdidas probabilísticas.

16.2 Errores frecuentes al interpretar un modelo

- Confundir **entrenamiento** con una evaluación independiente.
- Interpretar una probabilidad de softmax como certeza sobre la realidad.
- Ajustar hiperparámetros repetidamente contra el conjunto de prueba.
- Reportar una sola cifra sin distribución de errores, tamaño de muestra ni contexto.

17 Dónde se usan y qué límites tienen

Aplicaciones, precauciones y buenas prácticas

Las redes neuronales se emplean en visión por computador, procesamiento del lenguaje, predicción de series temporales, detección de patrones y muchas otras tareas. Sin embargo, su utilidad depende de la calidad del problema planteado y de los datos disponibles.

Sector	Aplicación ilustrativa	Qué debe verificarse
Industria	Detección de defectos en imágenes.	Iluminación, condiciones reales, falsos negativos.
Educación	Clasificación temática de documentos.	Etiquetas de calidad, idioma, generalización.
Salud	Apoyo a clasificación de imágenes médicas.	Evaluación clínica externa, sesgos, privacidad y supervisión profesional.
Servicios	Pronóstico de demanda.	Cambios temporales, deriva y estacionalidad.

ATENCIÓN

No existe una «red neuronal correcta» para todos los casos. Antes de elegir un modelo profundo, compara con un método base más simple: regresión logística, árboles o una heurística. Un modelo más complejo puede costar más y rendir igual o peor.

17.1 Tres preguntas para evaluar una propuesta

1. **Problema:** ¿qué decisión o estimación necesito y cuál es su costo de error?
2. **Datos:** ¿son suficientes, representativos, legales y pertinentes para el contexto?
3. **Evaluación:** ¿cómo mediré generalización, estabilidad e impactos indeseados?

LO ESENCIAL

El resultado de una red neuronal es una **estimación matemática** basada en los datos y el proceso de entrenamiento. No sustituye la validación específica del dominio ni la responsabilidad humana en decisiones de alto impacto.

18 Laboratorio guiado y ejercicios

Poner a prueba la comprensión

18.1 Ejercicio 1. Interpretación sin programación

Explica con palabras sencillas cuál es la diferencia entre peso, sesgo y función de activación. ¿Qué ocurre si cambian los pesos pero no las entradas?

18.2 Ejercicio 2. Cálculo de una neurona

Considera $(x_1, x_2) = (2, 3)$, $(w_1, w_2) = (0{,}5, -0{,}2)$ y $b = 0{,}1$. Calcula z y la salida ReLU. Comprueba el resultado con NumPy.

18.3 Ejercicio 3. Arquitecturas

¿Cuántos parámetros entrenables hay en un MLP con 3 entradas, una capa oculta de 5 neuronas y 2 salidas, incluyendo sesgos?

18.4 Ejercicio 4. Detectar un error de código

Una persona construyó una red que termina con `nn.Sigmoid()` y luego usa `nn.BCEWithLogitsLoss()`. ¿Qué debes modificar y por qué?

18.5 Ejercicio 5. Interpretación de métricas

Un clasificador detecta 30 positivos reales, deja sin detectar 20 y genera 10 falsas alarmas. Calcula precisión, sensibilidad y F_1.

18.6 Ejercicio 6. Miniinvestigación

En el notebook complementario, cambia las 16 neuronas ocultas por 2, 8 y 64. Compara desempeño en prueba y tiempo de entrenamiento, manteniendo las demás decisiones del experimento constantes. Evita elegir una arquitectura mirando repetidamente el resultado del conjunto de prueba.

Soluciones resumidas

1. Los pesos escalan cada entrada, el sesgo desplaza la combinación y la activación transforma la señal; cambiar pesos altera la predicción aunque las entradas permanezcan iguales.
2. $z = 0{,}5(2) - 0{,}2(3) + 0{,}1 = \boxed{0{,}5}$; ReLU$(z) = \boxed{0{,}5}$.
3. $3(5) + 5 + 5(2) + 2 = \boxed{32}$ parámetros.
4. Elimina `nn.Sigmoid()` y entrega logits a `BCEWithLogitsLoss`. Aplica sigmoide únicamente al interpretar probabilidades.

5. $VP = 30$, $FN = 20$, $FP = 10$: precisión $= 30/40 = \boxed{0{,}75}$, recall $= 30/50 = \boxed{0{,}60}$ y $F_1 = \boxed{0{,}667}$ aproximadamente.
6. No hay una única arquitectura ganadora: hay que examinar los resultados del experimento y considerar variación, tamaño de muestra y complejidad.

19 Glosario visual de referencia

Los términos que conviene dominar

Término	Significado sencillo
Neurona artificial	Unidad que combina entradas y entrega un valor transformado.
Peso / *weight*	Número ajustable que modula la influencia de una entrada.
Sesgo / *bias*	Número ajustable que desplaza la combinación lineal.
Capa oculta	Conjunto de neuronas situado entre entradas y salidas.
MLP	Perceptrón multicapa: red de capas densamente conectadas.
Logit	Puntuación de salida antes de convertirla en probabilidad.
Forward	Cálculo de la predicción desde las entradas hasta la salida.
Pérdida / *loss*	Número que representa el error según un criterio definido.
Gradiente	Vector de derivadas que describe cambios locales de la pérdida.
Backpropagation	Procedimiento eficiente para calcular gradientes mediante la regla de la cadena.
Optimizador	Algoritmo que ajusta los parámetros utilizando gradientes.
Tasa de aprendizaje	Hiperparámetro que controla el tamaño de los pasos de optimización.
Época	Recorrido completo por el conjunto de entrenamiento.
Minilote	Subconjunto de ejemplos procesado conjuntamente.
Overfitting / sobreajuste	El modelo aprende particularidades de entrenamiento y generaliza mal.
Data leakage / fuga	Información que no debería conocerse se filtra al entrenamiento.

19.1 La ruta de aprendizaje en seis preguntas

1. ¿Qué entradas recibe mi sistema?
2. ¿Cómo las transforma cada neurona y cada capa?
3. ¿Qué salida corresponde exactamente a mi tarea?
4. ¿Cómo cuantifico el error de esa salida?
5. ¿Cómo calculo gradientes y actualizo parámetros?
6. ¿Cómo compruebo que generaliza fuera del entrenamiento?

20 Bibliografía y material complementario

Origen, lecturas y recursos de práctica

Presentación base aportada. Héctor J. Hortúa, PhD. *Artificial Neural Networks*. Presentación incluida en el archivo «Redes Neuronales.docx» facilitado para preparar esta guía. La presentación se centra en definiciones, neuronas, funciones de activación, arquitecturas multicapa, softmax, descenso de gradiente, retropropagación y un ejercicio numérico de actualización de pesos.

Lecturas para profundizar.

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Capítulos de redes *feed-forward*, regularización y optimización. Disponible en `https://www.deeplearningbook.org/`.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Fundamentos probabilísticos y optimización.
- Documentación de PyTorch: `https://pytorch.org/tutorials/`.
- Guía de NumPy: `https://numpy.org/doc/stable/`.
- Documentación de scikit-learn: `https://scikit-learn.org/stable/`.

Notebook complementario `.ipynb`

Este material se acompaña de un cuaderno de práctica ejecutable con ejemplos de NumPy y PyTorch, cálculos de backpropagation, entrenamiento de una red y métricas. Puede abrirse en Jupyter Notebook, VS Code o Google Colab.

Nota editorial. Los diagramas se redibujaron para mejorar legibilidad; se mantuvieron los temas y la estructura de aprendizaje de la presentación de referencia, con explicaciones complementarias, código y distinciones técnicas añadidas. El documento no reproduce literalmente las diapositivas.